

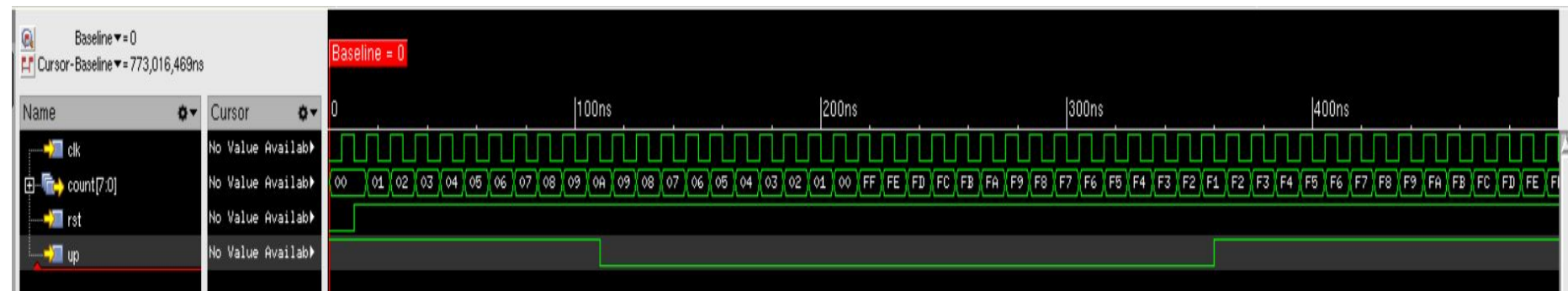


Assignment 1 & 2

Nabil KANA

ASIC design

Part 1: RTL Simulation



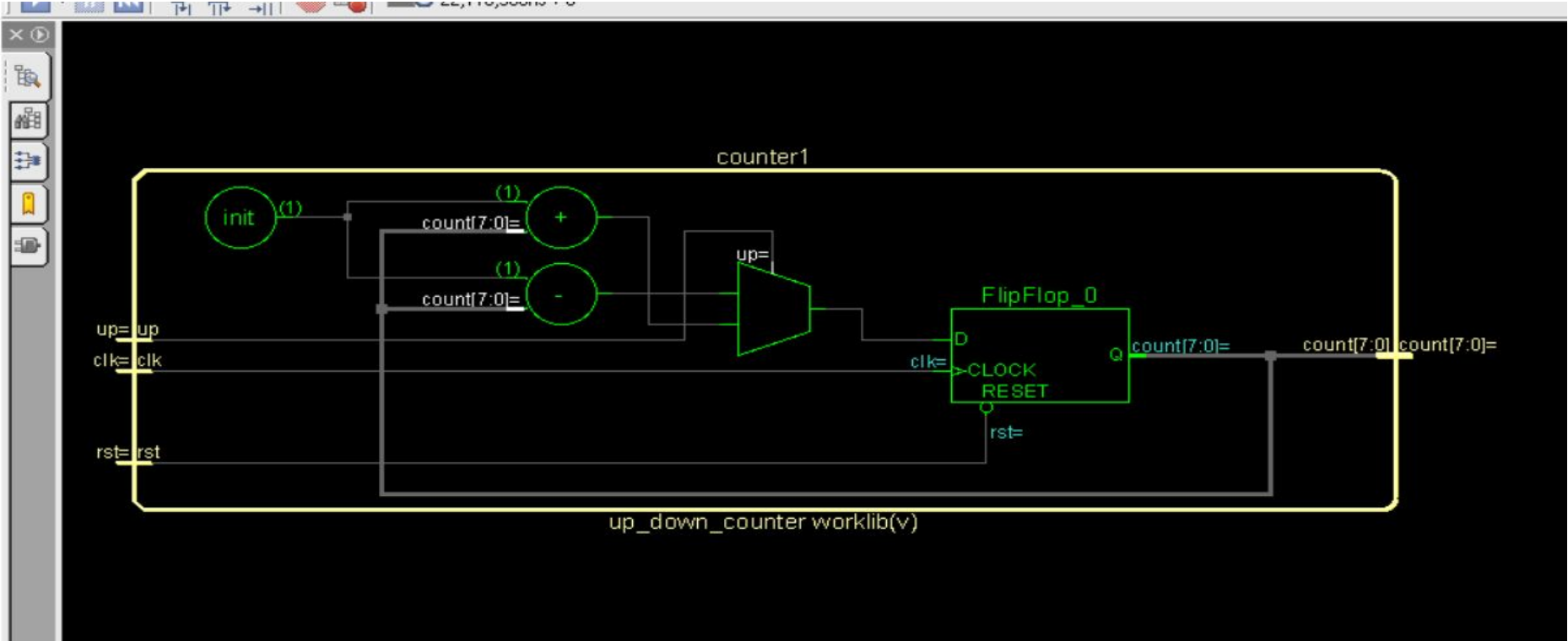
By adding an up input bit we can control whether the counter increments or decrements , we can see that when up=1 the counter increments , when it goes low , the counter starts decrementing until 00 and overflows to FF , by pulling up back again the counter goes up from F1,F2...

RTL and Testbench

```
module up_down_counter(clk,rst,up,count);  
input clk,rst,up;  
output reg [7:0] count;  
always@(posedge clk or negedge rst)  
begin  
if(!rst)  
count<=0;  
else  
begin  
if(up)  
count<=count+1;  
else  
count<=count-1;  
end  
end  
endmodule
```

```
module counter_test;  
reg clk, rst ,up;  
wire [7:0] count;  
initial  
begin  
clk=0;  
rst=0;  
up=1;  
#10;  
rst=1;  
#100;  
up=0;  
#250;  
up=1;  
end  
up_down_counter counter1(clk,rst,up, count);  
always #5 clk=~clk;  
endmodule
```

Schematic view of the design.



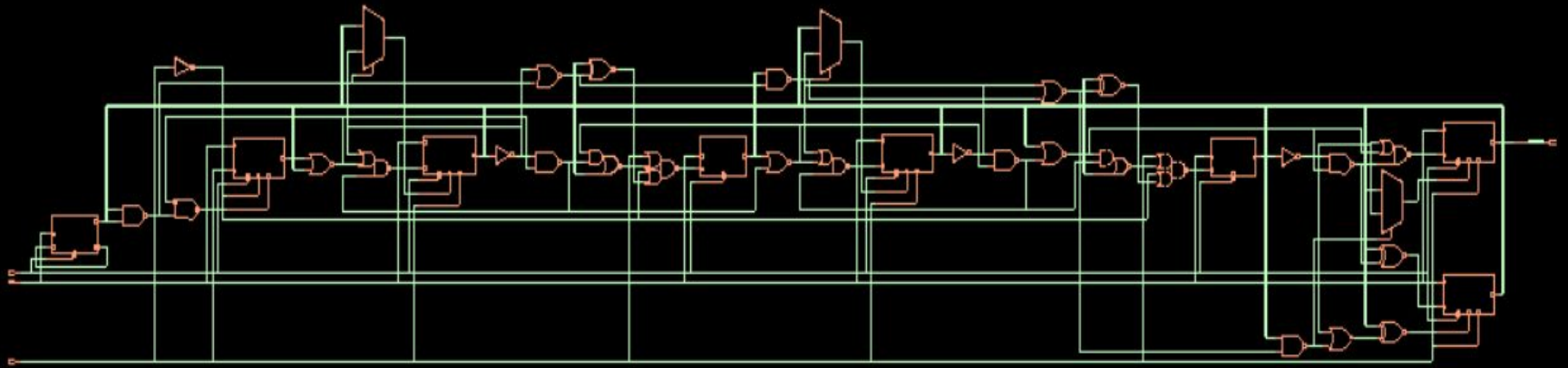
The RTL simulation verifies the *functional correctness* of the up-down counter. At this stage, the design is described behaviorally using arithmetic and control constructs such as $+$, $-$, and conditional statements. The simulator models these operations abstractly

Timing Constraints

```
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.01 [get_ports "clk"]
set_input_delay -max 1.0 [get_ports "rst"] -clock [get_clocks "clk"]
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]
set_input_delay -max 1.0 [get_ports "up"] -clock [get_clocks "clk"]
```

A constraint on “up” input was added for Part II: Synthesis

Part II: Synthesis



After synthesis , the RTL code is mapped to *standard cells* from the technology library. The resulting gate-level netlist replaces high-level arithmetic operations with combinational logic made of AND, OR, XOR, and MUX gates, along with D-flip-flops for storage.

The synthesized circuit represents the *actual hardware implementation* that would be realized on silicon

Screenshot of the generated netlist

```
module up_down_counter(clk, rst, up, count);
input clk, rst, up;
output [7:0] count;
wire clk, rst, up;
wire [7:0] count;
wire n_0, n_1, n_2, n_3, n_4, n_5, n_6, n_7;
wire n_8, n_9, n_10, n_11, n_12, n_13, n_14, n_15;
wire n_16, n_17, n_18, n_19, n_20, n_21, n_22, n_23;
wire n_24, n_25, n_26, n_27, n_28, n_29, n_30, n_31;
SDFFRHGX1 \count_reg[7] (.RN (rst), .CK (clk), .D (n_31), .SI (n_29),
    .SE (up), .Q (count[7]));
SDFFRHGX1 \count_reg[6] (.RN (rst), .CK (clk), .D (n_28), .SI (n_26),
    .SE (up), .Q (count[6]));
DFFRHGX1 \count_reg[5] (.RN (rst), .CK (clk), .D (n_30), .Q
    (count[5]));
XNOR2X1 g676_2398(.A (count[7]), .B (n_27), .Y (n_31));
OAI22X1 g681_5107(.A0 (up), .A1 (n_21), .B0 (n_19), .B1 (n_18), .Y
    (n_30));
XNOR2X1 g677_6260(.A (count[7]), .B (n_23), .Y (n_29));
SDFFRHGX1 \count_reg[4] (.RN (rst), .CK (clk), .D (n_17), .SI (n_14),
    .SE (up), .Q (count[4]));
OAI21X1 g679_4319(.A0 (n_25), .A1 (n_24), .B0 (n_27), .Y (n_28));
DFFRHGX1 \count_reg[3] (.RN (rst), .CK (clk), .D (n_20), .Q
    (count[3]));
MXI2XL g680_8428(.A (count[6]), .B (n_25), .S0 (n_22), .Y (n_26));
NAND2X1 g682_5526(.A (n_25), .B (n_24), .Y (n_27));
OR2XL g683_6783(.A (n_25), .B (n_22), .Y (n_23));
AOI21X1 g685_3680(.A0 (count[5]), .A1 (n_16), .B0 (n_24), .Y (n_21));
OAI22X1 g692_1617(.A0 (up), .A1 (n_11), .B0 (n_19), .B1 (n_9), .Y
    (n_20));
XNOR2X1 g686_2802(.A (count[5]), .B (n_15), .Y (n_18));
OAI21X1 g690_1705(.A0 (n_13), .A1 (n_12), .B0 (n_16), .Y (n_17));
SDFFRHGX1 \count_reg[2] (.RN (rst), .CK (clk), .D (n_7), .SI (n_5),
    .SE (up), .Q (count[2]));
NAND2X1 g688_5122(.A (count[5]), .B (n_15), .Y (n_22));
NOR2X1 g687_8246(.A (count[5]), .B (n_16), .Y (n_24));
MXI2XL g691_7098(.A (count[4]), .B (n_13), .S0 (n_10), .Y (n_14));
SDFFRHGX1 \count_reg[1] (.RN (rst), .CK (clk), .D (up), .SI (n_19),
    .SE (n_2), .Q (count[1]));
NAND2X1 g693_6131(.A (n_13), .B (n_12), .Y (n_16));
AOI21X1 g696_1881(.A0 (count[3]), .A1 (n_8), .B0 (n_12), .Y (n_11));
NOR2X1 g694_5115(.A (n_13), .B (n_10), .Y (n_15));
XNOR2X1 g697_7482(.A (count[3]), .B (n_6), .Y (n_9));
NOR2X1 g699_4733(.A (count[3]), .B (n_8), .Y (n_12));
OAI21X1 g701_6161(.A0 (n_4), .A1 (n_1), .B0 (n_8), .Y (n_7));
```

```
NAND2X1 g700_9315(.A (count[3]), .B (n_6), .Y (n_10));
MXI2XL g702_9945(.A (count[2]), .B (n_4), .S0 (n_3), .Y (n_5));
NOR2X1 g705_2883(.A (n_4), .B (n_3), .Y (n_6));
NAND2BX1 g704_2346(.AN (n_1), .B (n_3), .Y (n_2));
NAND2X1 g703_1666(.A (n_4), .B (n_1), .Y (n_8));
NOR2X1 g707_7410(.A (count[1]), .B (count[0]), .Y (n_1));
NAND2X1 g708_6417(.A (count[1]), .B (count[0]), .Y (n_3));
INVX1 g711(.A (count[4]), .Y (n_13));
INVX1 g712(.A (count[2]), .Y (n_4));
INVX1 g713(.A (up), .Y (n_19));
INVX1 g710(.A (count[6]), .Y (n_25));
DFFRX1 \count_reg[0] (.RN (rst), .CK (clk), .D (n_0), .Q (count[0]),
    .QN (n_0));
```

endmodule

Timing Report

```
=====
Generated by:      Genus(TM) Synthesis Solution 22.14-s059_1
Generated on:      Oct 08 2025  01:49:47 pm
Module:            up_down_counter
Operating conditions: PVT_0P9V_125C (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: MET (8677 ps) Late External Delay Assertion at pin count[0]
  Group: clk
  Startpoint: (R) count_reg[0]/CK
  Clock: (R) clk
  Endpoint: (R) count[0]
  Clock: (R) clk

      Capture          Launch
  Clock Edge: + 10000      0
  Src Latency: + 0         0
  Net Latency: + 0 (I)     0 (I)
  Arrival: = 10000        0

  Output Delay: - 1000
  Required Time: = 9000
  Launch Clock: - 0
  Data Path: - 323
  Slack: = 8677

Exceptions/Constraints:
  output_delay 1000 constraints_top.sdc_line_6_7_1

#-----#
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
#                                     (fF) (ps) (ps) (ps) Location
#-----#
count_reg[0]/CK -      -   R   (arrival)  8   -   100   0   0   (-,-)
count_reg[0]/Q  -      CK->Q R   DFFRX1    3  0.8  36  323  323  (-,-)
count[0]        <<<    -   R   (port)    -   -   -   0   323  (-,-)
#-----#
```

The up-down counter shows a very large positive slack of 8677 ns, which indicates that the circuit is extremely fast compared to the specified 10 ns clock period. This occurs because the design is small and simple — consisting of only basic logic (a few gates and flip-flops) — resulting in very short data path delays. Thus, we can increase the clock frequency (reduce the clock period) to better utilize the circuit's speed.

Power Report

Instance: /up_down_counter
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	1.28043e-09	2.58607e-06	4.64638e-08	2.63381e-06	83.19%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.03037e-09	2.83765e-07	1.17834e-07	4.02629e-07	12.72%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	1.29600e-07	1.29600e-07	4.09%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	2.31080e-09	2.86983e-06	2.93898e-07	3.16604e-06	100.00%
Percentage	0.07%	90.64%	9.28%	100.00%	100.00%

The synthesized *up-down counter* shows that switching power contributes about 9.28% of the total power, while leakage accounts for only 0.07%, with the remainder coming from internal power.

The clock network consumes around 4.09% of the total power, which is typical for a small sequential circuit.

Most of the power is spent in the combinational logic (adders and multiplexers) due to signal transitions.

Area Report

```
=====
Generated by:      Genus(TM) Synthesis Solution 22.14-s059_1
Generated on:      Oct 08 2025  01:50:26 pm
Module:           up_down_counter
Operating conditions: PVT_0P9V_125C (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====
```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
up_down_counter		39	106.362	0.000	106.362	<none> (D)
(D) = wireload is default in technology library						

The synthesized *up-down counter* consists of 39 cells, occupying a total cell area of 106.362. No additional area is reported for nets (net area = 0) since no wireload model was applied.

qor Report

```
Generated on:      Oct 08 2025   01:50:46 pm
Module:           up_down_counter
Operating conditions: PVT_0P9V_125C (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
```

Timing

Clock Period

clk 10000.0

Cost Group	Critical Path Slack	TNS	Violating Paths
clk	8677.4	0.0	0
default	No paths	0.0	
Total		0.0	0

Instance Count

```
Leaf Instance Count      39
Physical Instance count   0
Sequential Instance Count 8
Combinational Instance Count 31
Hierarchical Instance Count 0
```

Area

```
Cell Area                106.362
Physical Cell Area        0.000
Total Cell Area (Cell+Physical) 106.362
Net Area                  0.000
Total Area (Cell+Physical+Net) 106.362
```

```
Max Fanout               8 (clk)
Min Fanout                0 (rst)
Average Fanout            2.4
Terms to net ratio        3.3488
Terms to instance ratio   3.6923
Runtime                  53.779793 seconds
Elapsed Runtime           903 seconds
Genus peak memory usage   1807.07
Innovus peak memory usage no_value
```

The synthesized counter has a maximum fanout of 8, minimum of 0, and an average fanout of 2.4, indicating moderate loading on most signals. The terms-to-net ratio of 3.35 and terms-to-instance ratio of 3.69 suggest that nets typically connect a small number of pins relative to the number of cells, reflecting the simple structure of the design.

Screenshot of generated reports directory.

```
genus.cmd  genus.cmd2  genus.cmd4  genus_dft_script.tcl  genus.log1  genus.log3  genus.log5  outputs  reports
[nkqnn@kc-sse-tux01 synthesis]$ cd reports1
[nkqnn@kc-sse-tux01 reports1]$ ls
report_area.rpt  report_power.rpt  report_qor.rpt  report_timing.rpt
[nkqnn@kc-sse-tux01 reports1]$ pwd
/home/nkqnn/pd_fall_2025/Cadence-RTL-to-GDSII-Flow/counter_design_database_45nm/synthesis/reports1
[nkqnn@kc-sse-tux01 reports1]$
```